

AI Code Review & Security Analysis Report

Professional Code Quality & Security Assessment

Generated: 2026-08-14 09:54 UTC | Language: python | Model: llama-3.3-70b-versatile

Executive Summary

Code Health Score	Risk Score	Total Findings
0/100	45	12

Merge Readiness: unknown

PR summary generation was skipped because Groq rate limited the request.

Severity Breakdown

Critical	High	Medium	Low
9	3	0	0

Detailed Findings

Type	Severity	Line	Message	Recommendation
1. Hardcoded Password	● High	1	password = "admin123"	Store passwords in environment variables.
2. Hardcoded API Key	● High	3	api_key = "123456789"	Store secrets securely.
3. SQL Injection	● Critical	7	query = "SELECT * FROM users WHERE name='" + name + "'"	Use parameterized queries.
4. Dangerous Dynamic Execution	● Critical	9	eval(input())	Avoid dynamic code execution.
5. Dangerous Dynamic Execution	● Critical	11	exec("print('Hello')")	Avoid dynamic code execution.
6. Command Injection	● Critical	15	subprocess.run("dir", shell=True)	Do not use shell=True with user input.

Type	Severity	Line	Message	Recommendation
7. Security	● Critical	1	Hardcoded password	Use environment variables or a secure secrets management system to store sensitive data
8. Security	● Critical	3	Hardcoded API key	Use environment variables or a secure secrets management system to store sensitive data
9. Security	● Critical	5	SQL injection vulnerability	Use parameterized queries or prepared statements to prevent SQL injection
10. Security	● Critical	7	Use of eval function with user input	Avoid using eval function with user input, use a safer alternative such as ast.literal_eval
11. Security	● Critical	9	Use of exec function	Avoid using exec function, use a safer alternative such as importlib or subprocess with caution
12. Security	● High	11	Use of subprocess with shell=True	Use subprocess with caution and avoid using shell=True to prevent shell injection attacks

■ Key Recommendations

1. Address all Critical and High severity findings before merging.
2. Run additional security scanning tools to complement this analysis.
3. Consider implementing automated testing to catch similar issues in the future.

■ Code Quality Metrics

Overall Health Score: 0/100

Risk Level: Unknown

Maintainability: N/A

Security Score: N/A

■ Source Code Snapshot

```
password = "admin123"
```

```
api_key = "123456789"
```

```
name = input()
```

```
query = "SELECT * FROM users WHERE name='" + name + "'"
```

```
eval(input())
```

```
exec("print('Hello')")
```

```
import subprocess
```

```
subprocess.run("dir", shell=True)
```



This report was generated by the AI Code Review & Security Analysis Agent. Review all findings carefully before implementing changes.

Report generated on 2026-08-14 09:54 UTC